Домашнее задание №4

Page • Tag

Задание 1

На сервере была запущена сборка генома с помощью тула Velvet для разных размеров k-меров (31, 41, 51, 61). Для этого был запущен следующий bash-скрипт:

Bash Unwrap Copy

```bash
#!/bin/bash
#SBATCH --job-name=velvet_assembly
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH --output=/home/STUDY/FBMF/studfbmf02_10/hw4/velvet_%j.log
#SBATCH --partition=E5-2630-TITAN_X

VELVET=/home/STUDY/FBMF/bioinformatics/soft/velvet
R1=/home/STUDY/FBMF/studfbmf02_10/sem10/genome_de_novo/7_S4_L001_R1_001
.fastq
R2=/home/STUDY/FBMF/studfbmf02_10/sem10/genome_de_novo/7_S4_L001_R2_001
.fastq
OUTDIR=/home/STUDY/FBMF/studfbmf02_10/hw4

for K in 31 41 51 61; do
    echo "Запуск Velvet k=${K}"
    mkdir -p ${OUTDIR}/k${K}
    $VELVET/velveth ${OUTDIR}/k${K} ${K} -fastq -shortPaired -separate
$R1 $R2
    $VELVET/velvetg ${OUTDIR}/k${K} -ins_length 300
done
```

В итоге получили 4 сборки для соответствующих k-меров:

```
drwxr-xr-x.  2 studfbmf02_10 fbmf            8 May  4 19:59 k31
drwxr-xr-x.  2 studfbmf02_10 fbmf            8 May  4 19:59 k41
drwxr-xr-x.  2 studfbmf02_10 fbmf            8 May  4 19:59 k51
drwxr-xr-x.  2 studfbmf02_10 fbmf            8 May  4 19:59 k61
```

Задание 2

Для запуска сборки того же генома с помощью SPAdes для тех же размеров k-меров использовался скрипт:

```bash
#!/bin/bash
#SBATCH --job-name=spades
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=08:00:00
#SBATCH --output=/home/STUDY/FBMF/studfbmf02_10/hw4/spades_k_%j.log
#SBATCH --partition=E5-2630-TITAN_X

SPADES=/home/STUDY/FBMF/bioinformatics/soft/SPAdes-4.2.0-Linux/bin/spades.py
R1=/home/STUDY/FBMF/studfbmf02_10/sem10/genome_de_novo/7_S4_L001_R1_001.fastq
R2=/home/STUDY/FBMF/studfbmf02_10/sem10/genome_de_novo/7_S4_L001_R2_001.fastq
OUTDIR=/home/STUDY/FBMF/studfbmf02_10/hw4

for K in 31 41 51 61; do
    echo "Запуск SPAdes с k-mer = ${K}"
    mkdir -p ${OUTDIR}/spades_k${K}
    $SPADES \
        -1 $R1 \
        -2 $R2 \
        -o ${OUTDIR}/spades_k${K} \
        --threads $SLURM_CPUS_PER_TASK \
        --memory 32 \
        -k ${K}
done
```

Таким образом были получены сборки для SPAdes:

```
drwxr-xr-x.  7 studfbmf02_10 fbmf          20 May  4 19:59 spades_k31
drwxr-xr-x.  7 studfbmf02_10 fbmf          20 May  4 19:59 spades_k41
drwxr-xr-x.  7 studfbmf02_10 fbmf          20 May  4 19:59 spades_k51
drwxr-xr-x.  7 studfbmf02_10 fbmf          20 May  4 19:59 spades_k61
```

Далее был написан bash-скрипт для запуска quast:

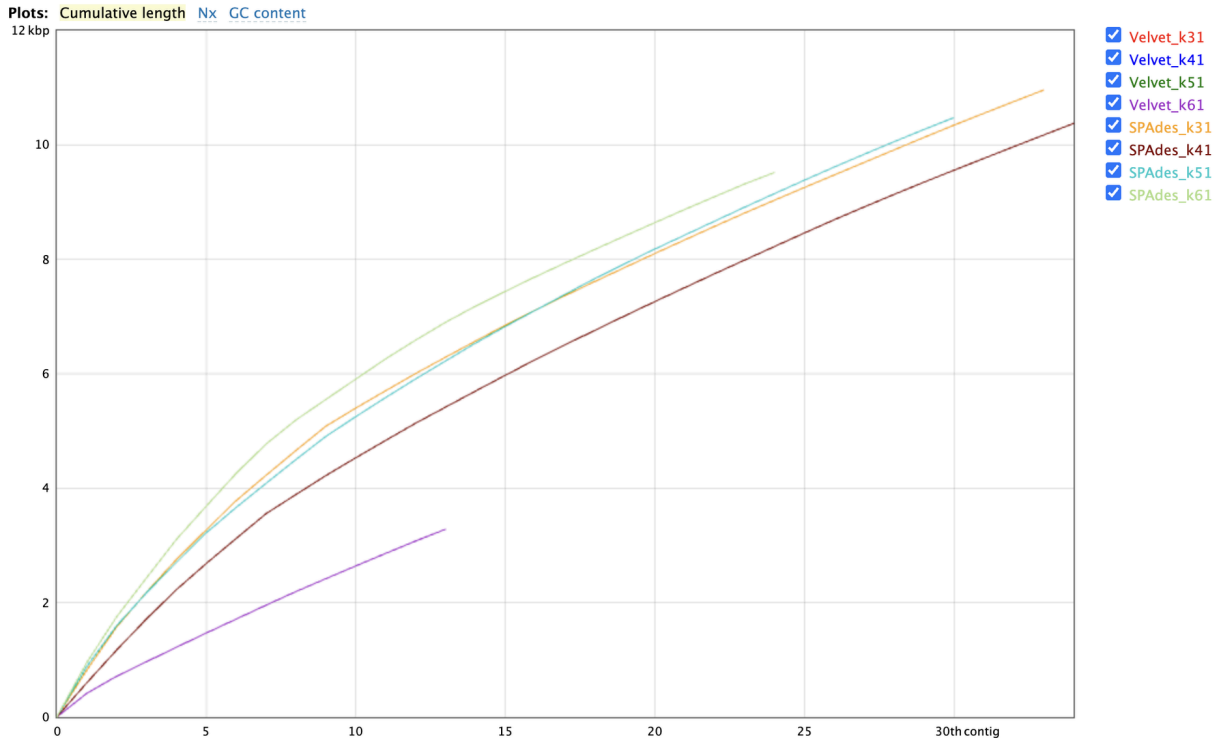
```bash
#!/bin/bash
#SBATCH --job-name=quast
#SBATCH --ntasks=1
```

```
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=08:00:00
#SBATCH --output=/home/STUDY/FBMF/studfbmf02_10/hw4/quast_%j.log
#SBATCH --partition=E5-2630-TITAN_X

QUAST=/home/STUDY/FBMF/bioinformatics/soft/bin/quast.py

VELVET_K31=/home/STUDY/FBMF/studfbmf02_10/hw4/k31/contigs.fa
VELVET_K41=/home/STUDY/FBMF/studfbmf02_10/hw4/k41/contigs.fa
VELVET_K51=/home/STUDY/FBMF/studfbmf02_10/hw4/k51/contigs.fa
VELVET_K61=/home/STUDY/FBMF/studfbmf02_10/hw4/k61/contigs.fa


SPADES_K31=/home/STUDY/FBMF/studfbmf02_10/hw4/spades_k31/contigs.fasta
SPADES_K41=/home/STUDY/FBMF/studfbmf02_10/hw4/spades_k41/contigs.fasta
SPADES_K51=/home/STUDY/FBMF/studfbmf02_10/hw4/spades_k51/contigs.fasta
SPADES_K61=/home/STUDY/FBMF/studfbmf02_10/hw4/spades_k61/contigs.fasta



OUTDIR=/home/STUDY/FBMF/studfbmf02_10/hw4/quast
python $QUAST \
    $VELVET_K31 \
    $VELVET_K41 \
    $VELVET_K51 \
    $VELVET_K61 \
    $SPADES_K31 \
    $SPADES_K41 \
    $SPADES_K51 \
    $SPADES_K61\
    -o $OUTDIR \
    --labels "Velvet_k31,Velvet_k41,Velvet_k51,Velvet_k61,SPAdes_k31,
SPAdes_k41, SPAdes_k51, SPAdes_k61" \
    --threads 8 \
    --min-contig 200
```

Минимальная длина контига для анализа была выставлена равной 200 bp, так как при сборке и Velvet и SPAdes получались очень маленькие контиги. В итоге была получена следующая таблица анализа по всем 8 сборкам:

Worst Median Best ☑ Show heatmap

Statistics without reference	Velvet_k31	Velvet_k41	Velvet_k51	Velvet_k61	SPAdes_k31	SPAdes_k41	SPAdes_k51	SPAdes_k61
# contigs	13	13	13	13	33	34	30	24
# contigs (>= 0 bp)	543	543	543	543	583	670	370	136
# contigs (>= 1000 bp)	0	0	0	0	0	0	0	0
# contigs (>= 5000 bp)	0	0	0	0	0	0	0	0
# contigs (>= 10000 bp)	0	0	0	0	0	0	0	0
# contigs (>= 25000 bp)	0	0	0	0	0	0	0	0
# contigs (>= 50000 bp)	0	0	0	0	0	0	0	0
Largest contig	414	414	414	414	817	597	882	949
Total length	3285	3285	3285	3285	10 960	10 376	10 478	9519
Total length (>= 0 bp)	52 070	52 070	52 070	52 070	40 778	52 924	37 708	21 306
Total length (>= 1000 bp)	0	0	0	0	0	0	0	0
Total length (>= 5000 bp)	0	0	0	0	0	0	0	0
Total length (>= 10000 bp)	0	0	0	0	0	0	0	0
Total length (>= 25000 bp)	0	0	0	0	0	0	0	0
Total length (>= 50000 bp)	0	0	0	0	0	0	0	0
N50	244	244	244	244	303	281	345	512
N90	216	216	216	216	219	214	231	231
auN	263.3	263.3	263.3	263.3	410.2	346.5	419.4	502.2
L50	6	6	6	6	11	13	10	7
L90	12	12	12	12	28	29	26	20
GC (%)	46.33	46.33	46.33	46.33	45.04	45.43	45.17	44.54
Per base quality								
# N's per 100 kbp	0	0	0	0	0	0	0	0
# N's	0	0	0	0	0	0	0	0



Velvet: все четыре сборки (k31–k61) дали одинаковый результат - 543 контига суммарной длиной 3285 bp среди контигов ≥200 bp. Это говорит о том, что Velvet в данном случае практически не зависит от выбора k-мера в диапазоне 31–61, и сборка получилась очень фрагментированной и неполной. Максимальный контиг имеет длину всего 414 bp, N50 = 244 bp, что очень мало. Скорее всего параметры сборки были не оптимальны, либо данные плохо подходят под этот алгоритм.

SPAdes: результаты значительно лучше и варьируются в зависимости от размера k-мера. Лучшей сборкой оказался SPAdes k61 (наибольший контиг 949 bp, N50 = 512 bp, L50 = 7). SPAdes k51 показал второй результат по N50 (345 bp) и наибольшую суммарную длину сборки (10478 bp).

Лучшие результаты SPAdes по сравнению с Velvet объясняется тем, что SPAdes использует граф де Брёйна с несколькими k-мерами одновременно, тогда как Velvet работает с одним фиксированным k и более чувствителен к его выбору. При этом ни одна из сборок не даёт контигов длиннее 1000 bp, что говорит о недостаточной глубине покрытия или сложности генома.

Задание 3

Теперь попробуем улучшить сборки для SPAdes и Velvet

Для Velvet:

- Будем использовать оптимальный k-мер (так как все сборки получились одинаковыми, то возьмём k=51)
- Добавим `-cov_cutoff auto` в `velvetg` - отсекает малопокрытые контиги
- Добавим `-min_contig_lgth 200` - убирает очень короткие контиги-мусор

Bash

```bash
#!/bin/bash
#SBATCH --job-name=velvet_assembly
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH --output=/home/STUDY/FBMF/studfbmf02_10/hw4/velvet_%j.log
#SBATCH --partition=E5-2630-TITAN_X

VELVET=/home/STUDY/FBMF/bioinformatics/soft/velvet
R1=/home/STUDY/FBMF/studfbmf02_10/sem10/genome_de_novo/7_S4_L001_R1_001
.fastq
R2=/home/STUDY/FBMF/studfbmf02_10/sem10/genome_de_novo/7_S4_L001_R2_001
.fastq
OUTDIR=/home/STUDY/FBMF/studfbmf02_10/hw4

# Улучшенная сборка Velvet с k=51
echo "Запуск улучшенного Velvet k=51"
mkdir -p ${OUTDIR}/k51_improved
$VELVET/velveth ${OUTDIR}/k51_improved 51 -fastq -shortPaired -separate
$R1 $R2
$VELVET/velvetg ${OUTDIR}/k51_improved \
    -ins_length 300 \
    -cov_cutoff auto \
    -min_contig_lgth 200
```

Для SPAdes:

- Поставим `-k auto`, чтобы алгоритм сам подбирал оптимальное значение k-мера
- Добавим `--careful` - включает дополнительную коррекцию ошибок через BayesHammer
- Добавим `--cov-cutoff auto` - отсекает малопокрытые контиги

```bash
#!/bin/bash
#SBATCH --job-name=spades_improved
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=08:00:00
#SBATCH --output=/home/STUDY/FBMF/studfbmf02_10/hw4/spades_improved_%j.log
#SBATCH --partition=E5-2630-TITAN_X

SPADES=/home/STUDY/FBMF/bioinformatics/soft/SPAdes-4.2.0-Linux/bin/spades.py
R1=/home/STUDY/FBMF/studfbmf02_10/sem10/genome_de_novo/7_S4_L001_R1_001.fastq
R2=/home/STUDY/FBMF/studfbmf02_10/sem10/genome_de_novo/7_S4_L001_R2_001.fastq
OUTDIR=/home/STUDY/FBMF/studfbmf02_10/hw4

# Улучшенная сборка SPAdes с автоподбором k-мера
echo "Запуск улучшенного SPAdes"
mkdir -p ${OUTDIR}/spades_improved
$SPADES \
    -1 $R1 \
    -2 $R2 \
    -o ${OUTDIR}/spades_improved \
    --threads $SLURM_CPUS_PER_TASK \
    --memory 32 \
    -k auto \
    --careful \
    --cov-cutoff auto
```

После выполнения анализа quast, были получены следующие результаты:

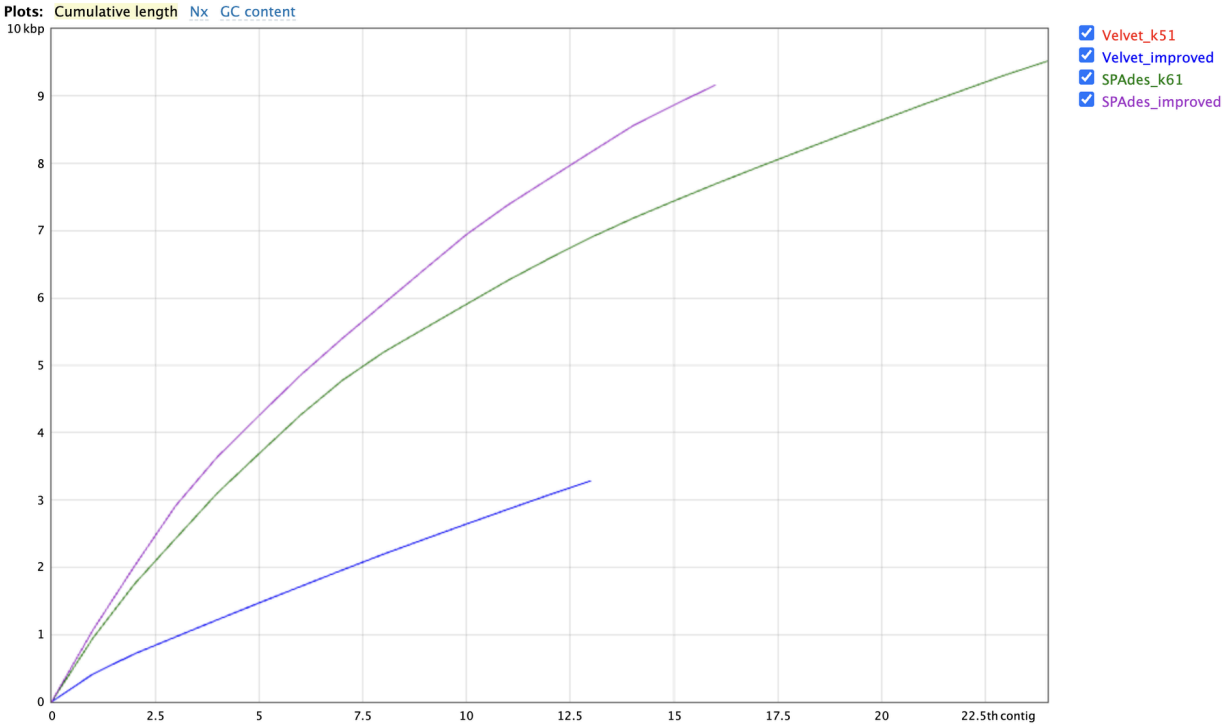
```bash
#!/bin/bash
#SBATCH --job-name=quast
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=08:00:00
#SBATCH --output=/home/STUDY/FBMF/studfbmf02_10/hw4/quast_%j.log
#SBATCH --partition=E5-2630-TITAN_X

QUAST=/home/STUDY/FBMF/bioinformatics/soft/bin/quast.py

VELVET_K51=/home/STUDY/FBMF/studfbmf02_10/hw4/k51/contigs.fa
VELVET_IMPROVED=/home/STUDY/FBMF/studfbmf02_10/hw4/k51_improved/contigs
.fa

SPADES_K61=/home/STUDY/FBMF/studfbmf02_10/hw4/spades_k61/contigs.fasta
SPADES_IMPROVED=/home/STUDY/FBMF/studfbmf02_10/hw4/spades_improved/cont
igs.fasta

OUTDIR=/home/STUDY/FBMF/studfbmf02_10/hw4/quast_final

python $QUAST \
    $VELVET_K51 \
    $VELVET_IMPROVED \
    $SPADES_K61 \
    $SPADES_IMPROVED \
    -o $OUTDIR \
    --labels "Velvet_k51,Velvet_improved,SPAdes_k61,SPAdes_improved" \
    --threads 8 \
    --min-contig 200
```

Для улучшения сборки Velvet добавляем `-cov_cutoff auto`, который автоматически отсекает контиги с низким покрытием, они скорее всего являются ошибками сборки, а не реальными геномными последовательностями. Для SPAdes используем `-k auto` чтобы алгоритм сам подобрал оптимальный набор k-меров под длину ридов, `--careful` для дополнительной коррекции ошибок секвенирования, и `--cov-cutoff auto` для фильтрации артефактов.

Worst Median Best

☑ Show heatmap

Statistics without reference	Velvet_k51	Velvet_improved	SPAdes_k61	SPAdes_improved
# contigs	13	13	24	16
# contigs (>= 0 bp)	543	13	136	38
# contigs (>= 1000 bp)	0	0	0	1
# contigs (>= 5000 bp)	0	0	0	0
# contigs (>= 10000 bp)	0	0	0	0
# contigs (>= 25000 bp)	0	0	0	0
# contigs (>= 50000 bp)	0	0	0	0
Largest contig	414	414	949	1069
Total length	3285	3285	9519	9167
Total length (>= 0 bp)	52 070	3285	21 306	12 103
Total length (>= 1000 bp)	0	0	0	1069
Total length (>= 5000 bp)	0	0	0	0
Total length (>= 10000 bp)	0	0	0	0
Total length (>= 25000 bp)	0	0	0	0
Total length (>= 50000 bp)	0	0	0	0
N50	244	244	512	600
N90	216	216	231	387
auN	263.3	263.3	502.2	659.2
L50	6	6	7	6
L90	12	12	20	14
GC (%)	46.33	46.33	44.54	43.81
Per base quality				
# N's per 100 kbp	0	0	0	0
# N's	0	0	0	0



Velvet improved vs Velvet k51: улучшенный Velvet показал идентичные результаты по метрикам качества (N50, N90, largest contig), однако `-cov_cutoff auto` убрал все низкопокрытые контиги, число контигов (≥0 bp) сократилось с 543 до 13. Таким образом улучшение Velvet дало более чистую сборку, но не более длинные контиги, что говорит о принципиальном ограничении алгоритма на этих данных.

SPAdes improved vs SPAdes k61: улучшенный SPAdes показал реальное улучшение качества сборки. N50 вырос с 512 до 600 bp, N90 вырос с 231 до 387 bp, появился первый контиг длиннее 1000 bp (1069 bp), число контигов сократилось с 136 до 38, auN вырос с 502.2 до 659.2. Это результат совместного действия `-k auto` (оптимальный набор k-меров), `--careful` (коррекция ошибок) и `--cov-cutoff auto` (фильтрация мусора).

SPAdes во всех вариантах превосходит Velvet. Лучшая сборка SPAdes improved (наибольший контиг, наибольший N50 и N90, наименьшее число контигов среди SPAdes-сборок). Velvet принципиально ограничен одним k-мером и отсутствием встроенной коррекции ошибок, поэтому даже с оптимизацией параметров не может конкурировать со SPAdes.